考点精析

贪心算法与树

第一部分贪心算法

本章目标 会识别、会设计、会证明、会避坑。重点不是“每次选最好的”这句话，而是能说明所选的局部最优为什么可扩展为全局最优。

1.1核心概念

贪心算法在每一步都作出当前看来最优的、不可撤销的选择，希望由这些局部最优选择构成全局最优解。它通常不枚举所有方案，也不回退。

- 贪心选择性质：存在一个全局最优解，其中包含当前作出的贪心选择。
- 最优子结构：作出贪心选择后，剩余问题的最优解可以与当前选择组合成原问题的最优解。
- 无后效性：后续决策只依赖当前剩余状态，不需要重新考虑已经确定的选择。

1.2贪心、动态规划与回溯的比较

方法	决策特点	是否回退	典型适用条件	典型问题
贪心	每步作局部最优选择	否	贪心选择性质+最优子结构	活动选择、MST、哈夫曼
动态规划	保存并组合子问题最优值	不以搜索回退为核心	最优子结构+重叠子问题	0/1背包、LCS
回溯	枚举候选并剪枝	是	解空间可系统搜索	排列、N皇后

高频判断 “有最优子结构”并不等于“可用贪心”。动态规划也要求最优子结构；贪心额外要求某个局部选择可安全地首先作出。

1.3标准解题与证明模板

1. 定义候选集、约束与目标函数，明确要最大化或最小化什么。
2. 提出贪心规则，并确定实现该规则所需的排序或优先队列。
3. 证明贪心选择性质： 常用交换论证、领先法或割/环性质。
4. 证明剩余问题与原问题同型，满足最优子结构。
5. 给出算法、复杂度与边界情况。

交换论证模板 设O 为某个最优解，G 为贪心算法首先选择的元素。若G 不在O 中，找到O 中 与G 对应的元素x，证明用G 替换x 后仍然可行且目标值不变差，于是得到一个包含G 的最优解。随后对剩余子问题归纳。

1.4通用代码框架

```
sort(items.begin(),items.end(), cmp);//  让“局部最优”可被顺序取得
Answer ans;
for(const    auto&x:items){
  if (feasible(ans,x)){          // 加入后仍满足约束
     ans.add(x);                 //选择不可撤销



return ans;
```

复杂度通常由排序主导，为O(n logn)；如果输入已经有序，则扫描部分常为O(n)。使用优先队列时通常为 O(n log n) 或O(V+E)logV)。

1.5典型考题精析：活动选择

典型题1 | 选择最多个互不冲突活动

题目给定活动区间(1, 4)、(3, 5)、(0, 6)、(5, 7)、(3, 9)、(5, 9)、(6, 10)、(8, 11)、(8, 12)、(2, 14)、(12, 16)，求最多能选多少个两两兼容的活动。

精析

6. 按结束时间从小到大排序。 首选结束最早的(1, 4)，它为后续留下最大的时间空间。
7. 从前向后扫描，只要下一个活动的开始时间不早于当前已选活动的结束时间就选择。
8. 依次可选(1, 4)、(5, 7)、(8, 11)、(12, 16)。其他选择方案不会得到更多活动。
9. 证明可用交换论证：把任一最优解的第一个活动替换为结束时间更早的贪心活动，不会减少后续可选范围。

答案 最多选择4个活动，一组最优解为{(1, 4), (5, 7), (8, 11), (12, 16)}。

复杂度：排序O(n log n), 扫描O(n), 总复杂度O(n log n)。

易错提醒 区间是否允许端点相接必须看题意。若(s,f) 模型允许相接，判断为s ≥lastFinish; 若闭区间且端点重合算冲突，则用s>lastFinish。

```
sort(a.begin(),a.end(,D(const Activity&x,const Activity&y){
   return x.finish<y.finish;
);
int count=0,lastFinish =INT_MIN;
for(auto  &x:a){
   if(x.start>=lastFinish){
```

```
++count;
lastFinish    =x.finish;
```

1.6典型考题精析：部分背包

典型题2 | 单位价值排序

题目背包容量为50,三件物品的(重量，价值)分别为(10,60)、(20,100)、(30,120)。物品可以切分，求最大价值。

精析

10. 计算单位重量价值：6、5、4。
11. 按单位价值从高到低装入： 先取第一件，剩余容量40;再取第二件，剩余容量20。
12. 第三件只能取20/30,其贡献为120×20/30=80。

答案最大价值为60+100+80=240。

复杂度：排序O(n logn),扫描O(n)。

易错提醒 若物品不可切分，这是0/1背包，按单位价值贪心可能失败。例如容量50时，本题0/1最优为第二、三件，总价值220;具体题目必须先确认能否切分。

1.7典型考题精析：哈夫曼编码

哈夫曼算法每次合并权值最小的两棵树，新结点权值为二者之和，再将新结点放回候选集合。用最小堆可高效完成。

典型题3 | 构造最优前缀编码树

题目字符A、B、C、D、E、F 的频率分别为5、9、12、13、16、45,求哈夫曼树的带权路径长度WPL。

精析

13. 取5和9合并为14;候选为12、13、14、16、45。
14. 取12和13合并为25;候选为14、16、25、45。
15. 取14和16合并为30;候选为25、30、45。
16. 取25和30合并为55;候选为45、55。
17. 取45和55合并为100。所有非根合并权之和等于WPL。

答案 WPL=14+25+30+55+100=224。

复杂度：使用最小堆为O(n log n),空间O(n)。

易错提醒 左右孩子交换只会改变具体0/1编码，不改变各字符码长和WPL。若权值相同，哈夫曼树可能不唯一，但 WPL 相同。

1.8典型考题精析：最小生成树

最小生成树（MST）要求图连通、无向、带权，目标是选V-1 条边连接全部顶点且总权值最小。边权可以为负；若图不连通，只能得到最小生成森林。

算法	贪心选择	常用结构	复杂度	更适合
Kruskal	按权值从小到大选不成环的边	排序+并查集	O(E log E)	稀疏图
Prim	从当前树向外选最小跨边	邻接表+最小堆	O(E logV)	稠密图或从点扩展

典型题4| Kruskal 手算

题目无向图有边AB=1、BC=2、AC=3、BD=4、CD=5、CE=6、DE=7，求一棵MST的边集与总权值。

精析

18. 按权值排序：AB、BC、AC、BD、CD、CE、DE。
19. 选AB、BC;AC 会与AB、BC 构成环，跳过。
20. 选 BD; CD 会构成环，跳过。
21. 选CE。此时5个顶点已由4条边连通，结束。

答案 MST 边集可为{AB,BC,BD,CE}，总权值1+2+4+6=13。

复杂度：排序O(E log E),并查集近似O(Eα(V)。

易错提醒 Kruskal的停止条件是已选V-1 条边，不是把所有边都扫描完。遇到等权边时 MST 可能不唯

1.9典型考题精析：Dijkstra

Dijkstra 每次确定当前未确定顶点中距离源点最小者，并用它松弛邻边。其贪心正确性依赖边权非负： 一旦某点被确定，后续绕路不可能再把距离降得更小。

```
vector<long long>dist(n,INF);
priority_queue<pair<ll,int>,vector<pair<ll,int>>,greater<pair<ll,int>>>pq;
dist[s]=0;
```

```
Pq.push({0,s});
while  (pq.empty0){
  auto   [d,u]=pq.top();pq.pop();
  if(d!= dist[u])continue;       //  跳过过期条目
  for (auto [v,w]:g[u]){
    if(dist[v]>d+w){
      dist[v]=d+w;
      Pq.push({dist{y),v});
```

适用边界 Dijkstra 不能直接处理负权边；有负边且无负环时考虑Bellman-Ford。 若只求无权图最短路，用 BFS。

复杂度：用二叉堆维护未确定顶点，Dijkstra 时间复杂度为 O((V+E)log V)，空间 O(V)；若用朴素数组找最小，则为 O(V^2)。

结果：s→t 最短距离为 6。路径恢复：从 t 沿 prev 回溯，prev[t]=a、prev[a]=b、prev[b]=s，得路径 s→b→a→t。

第4轮：选 t(6)。全部顶点确定，结束。

第3轮：选 c(6)。松弛：t 经 c 为 6+2=8>6，不更新。

第2轮：选 a(5)。松弛：t 经 a 为 5+1=6，dist[t]=6、prev[t]=a。

第1轮：选 dist 最小的 b(2)。松弛：a 经 b 为 2+3=5<7，dist[a]=5、prev[a]=b；c 经 b 为 2+4=6，dist[c]=6、prev[c]=b。

初始化：S=∅，dist=(s:0，a:7，b:2，c:∞，t:∞)，prev=(—，s，s，—，—)

维护三个结构：S（已确定最短距离的顶点集）、dist（当前最短距离）、prev（前驱，用于路径恢复）。

例题：求从源点 s 出发到各顶点的最短路径。无向图边权：s→b=2，s→a=7，a→b=3，a→t=1，b→c=4，c→t=2。

典型题精析：Dijkstra 手算与路径恢复

1.10高频错误清单

- 看到“最优”就直接写贪心， 没有验证贪心选择性质。
- 排序关键字选错：活动选择应优先最早结束，而非最早开始或持续时间最短。
- 把部分背包策略套用到0/1背包。
- MST 忘记图必须是无向连通图；把最短路径树与最小生成树混为一谈。
- Dijkstra 忽略负边，或优先队列中不跳过过期状态。
- 只写算法结果，不写复杂度、适用条件与正确性说明。

第二部分 树

本章目标 会算性质、会写遍历、会由序列重建、会维护BST/AVL/ 堆的不变量，并能在手算题中清晰画出每一步结构变化。

2.1基本术语与通用性质

术语	含义	考点提醒
根	没有父结点的唯一结点	非空树只有一个根
叶子	度为0的结点	不要把“没有左孩子”误判为叶子
结点的度	该结点的孩子数	树的度是所有结点度数的最大值
深度	从根到该结点的边数(常从0开始)	有些教材把根所在层记为1,需按题目口径

术语	含义	考点提醒
高度	从结点到最深叶子的最长边数	空树、单结点树的高度定义可能因教材不同
森林	若干棵互不相交的树	删除树根后得到由各子树组成的森林

- 边数公式：含 n 个结点的非空树恰有n-1 条边。
- 度数求和：所有结点的度数之和等于边数n-1。
- 一般树计数： 若 ni 表示度为i 的结点数，则n=∑ni, 且n-1=∑i·ni。

2.2二叉树高频性质

性质	公式/结论	使用条件
第i层最大结点数	2^(i-1)	根为第1层
高度为h的二叉树最大结点数	2^h-1	高度按层数h计算
非空二叉树叶子数	no =n2+1	no 为 度 0 ，n2为度2
完全二叉树结点编号	父L i/2」;左2i;右2i+1	1-based数组
完全二叉树结点编号	父L(i-1)/2」;左2i+1;右2i+2	0-based数组
n结点完全二叉树高度	L logzn」+1	根为第1层

典型题5 | 由度数求叶子结点数

题目一棵非空二叉树中，度为2的结点有20个，度为1的结点有7个，求叶子结点数和总结点数。

精析

22. 应用二叉树恒等式no =n2 +1，得到no =21。

23. 总结点数n=no +n1 +n2 =21+7+20=48。

答案 叶子结点21个，总结点48个。

易错提醒 no =n2 +1 与n1 无关。不要误写成no =n1 +n2。

2.3二叉树遍历

遍历	访问顺序	典型用途
前序	根 → 左 → 右	复制树、输出前缀表达式、由前序定位子树根
中序	左→根→右	BST得到有序序列
后序	左 → 右 → 根	释放子树、计算目录大小、后缀表达式
层序	逐层从左到右	最短层数、完全二叉树检查；使用队列

```
void  preorder(Node*x){
   if (lx)return;
   visit(x);
   preorder(x->left);
   preorder(x->right);


void   inorder(Node*x){
   if (lx)return;
   inorder(x->left);
   visit(x);
   inorder(x->right);


void   postorder(Node*x){
   if(lx)return;
   postorder(x->left);
   postorder(x->right);
   visit(x);
```

> 记忆方法 前/中/后的“前中后”指根结点相对于左右子树的访问位置；左右子树的相对次序通常始终是先左后右。

2.4非递归遍历模板

```
//中序遍历：一路压左，弹出访问，再转向右子树
stack<Node*>st;
Node*cu  r=  root;
while(cur     ||Ist.empty(){
   while(cur){
      st.push(cur);
      cur  =cur->left;

   cur      =st.top();st.pop();
```

```
    visit(cur);
    cur= cur->right;


//层序遍历：队列先进先出
queue<Node*>q;
if(root)q.push(root);
while (lq.empty(){
    Node*u=q.front();q.pop();
    visit(u);
    if (u->left)q.push(u->left);
    if(u->right)q.push(u->right);
```

2.5由遍历序列重建二叉树

在结点值互不相同的前提下，前序+中序或后序+中序能唯一确定一棵二叉树。仅有前序+后序通常不能唯一确定，因为无法判断只有一个孩子时它是左孩子还是右孩子。

典型题6 | 前序与中序重建

题目已知前序序列为ABDECF, 中序序列为DBEACF, 求后序序列。

精析

24. 前序首元素A是根。A在中序中左侧DBE属于左子树，右侧CF属于右子树。

25. 左子树前序为BDE, 中序为DBE, 所以B为根，D、E分别为左右孩子。

26. 右子树前序为CF, 中序为CF, 所以C为根且F为右孩子。

27. 按左—右—根写后序：左子树DEB; 右子树FC; 最后根A。

答案后序序列为DEBFCA。

复杂度：若使用哈希表记录中序下标，重建时间O(n)、递归栈O(h)。

易错提醒 分割前序区间时，左子树长度必须由中序中的根位置计算；不要凭图形直觉截断。

2.6二叉搜索树 (BST)

BST的不变量：对任一结点x, 其左子树所有关键字小于x, 右子树所有关键字大于x（重复键如何处理需事先约定）。因此中序遍历得到非递减序列。

操作	核心过程	复杂度
查找	按关键字大小向左或向右下降	O(h)
插入	查找失败位置作为新叶子	O(h)

操作	核心过程	复杂度
删除叶子	直接移除	O(h)
删除单孩子结点	用唯一孩子接替	O(h)
删除双孩子结点	用中序后继/前驱替换，再删除该后继/前驱	O(h)

典型题7 | BST 插入与删除

题目依次插入50、30、70、20、40、60、80，然后删除50。写出一种合法结果的根及中序序列。

精析

28. 50有两个孩子，不能直接把某一棵子树整体上移而丢失另一棵。

29. 选择中序后继，即右子树中的最小结点60，用60替换50。

30. 原位置的60是叶子，随后将其删除。

31. BST 不变量保持不变，因此中序序列仍然有序。

答案 一种结果以60为根；中序序列为20、30、40、60、70、80。也可用前驱40替换，得到另一种合法 BST。

复杂度：O(h)；平均O(log n)，最坏退化链为O(n)。

易错提醒 “BST 操作总是O(log n)”是错误表述。只有树高为Olog n)时才成立；普通BST 可能退化。

2.7 AVL 平衡二叉搜索树

AVL 树首先是BST，并要求每个结点的平衡因子BF= 左子树高度−右子树高度属于{−1, 0, 1}。插入或删除后，沿祖先链更新高度并在第一个失衡点旋转。

失衡类型	插入方向	修复方式	一句话判断
LL	失衡结点的左孩子的左侧	对失衡结点右旋	外侧左左→单右旋
RR	失衡结点的右孩子的右侧	对失衡结点左旋	外侧右右→单左旋
LR	失衡结点的左孩子的右侧	先左旋左孩子，再右旋失衡点	内侧左右→先左后右
RL	失衡结点的右孩子的左侧	先右旋右孩子，再左旋失衡点	内侧右左→先右后左

复杂度：查找被删结点 O(log n)，向上更新与旋转最多沿祖先链 O(log n) 层、每层 O(1)，总时间 O(log n)。

例：删除叶结点使某祖先左子树高度减 1 而右子树不变，该祖先平衡因子变为 +2，按 LL 或 LR 修复；修复后若该祖先的父结点再次失衡，继续向上重复，直到根。

旋转类型（LL、RR、LR、RL）与插入完全相同：失衡结点与其孩子、孙子处于同侧外侧时用单旋（LL 右旋、RR 左旋），处于两侧内侧时用双旋（LR 先左后右、RL 先右后左）。

删除结点后，被删结点到根的路径上各子树高度可能改变，需要沿该路径向上更新高度，并在每个结点重新检查平衡因子。与插入不同：插入在第一个失衡点旋转修复后通常即可停止；删除则可能使修复点的祖先重新失衡，必须继续向上检查，直到到达根或不再失衡为止。

AVL 删除：沿祖先链连续再平衡

典型题8 | AVL的 LR 旋转

题 目 向 空AVL 树依次插入30、10、20，说明失衡类型与旋转结果。

精析

32. 插入20后，30的左子树高度比右子树高2，因此30失衡。

33. 新结点位于30的左孩子10的右侧，属于LR 型。

34. 先对10左旋，使20成为该局部结构上方结点；再对30右旋。

答案 旋转后20为根，10为左孩子，30为右孩子。

复杂度：单次旋转 O(1)，插入查找和向上更新共O(log n)。

易错提醒 旋转名称按“从失衡点到新插入点的方向”判断，不是按最终旋转方向命名。

2.8堆与优先队列

堆是一棵完全二叉树。最大堆要求父结点关键字不小于孩子，最小堆相反。堆只保证父子之间的偏序，不保证中序或层序整体有序。

操作	方法	复杂度
访问堆顶	读取数组首元素	0(1)
插入	末尾加入后上浮	0logn)
删除堆顶	末元素移到根后下沉	0logn)
自底向上建堆	从最后一个非叶结点开始下沉	0(n)
堆排序	反复交换堆顶与末尾并缩小堆	0(n logn)

对比：若逐个插入元素（每个新元素从底部上浮）建堆，代价为 O(n log n)；自底向上建堆避免了这一开销，是堆排序初始化阶段的标准做法。

其中 Σ_{h≥0} h/2^h = 2 收敛，故建堆为 O(n)。直觉上，大多数结点位于底层、下沉距离很短，因此总代价是线性的。

Σ_{h=0}^{⌊log n⌋} ⌈n/2^(h+1)⌉ · h ≤ n · Σ_{h≥0} h/2^(h+1) = O(n)，

复杂度：高度为 h 的结点下沉至多 h 次、每次 O(1)；高度为 h 的结点至多 ⌈n/2^(h+1)⌉ 个，总代价过程：对数组 a[0..n-1]，从最后一个非叶结点下标 ⌊n/2⌋−1 开始，向前依次对每个下标执行下沉（heapify），直至根。

自底向上建堆 O(n) 的过程与证明

典型题9 | 最大堆删除堆顶

题目 最大堆的层序数组为[90, 70, 80, 20, 60, 50, 40]，删除堆顶后写出调整结果。

精析

35. 将末元素40移到根，临时数组为[40, 70, 80, 20, 60, 50]。

36. 比较40的两个孩子70、80，应与更大的80交换。

37. 40到达下标2 (0- based) ， 其孩子只有50；再与50交换。

答案 调整后最大堆为[80, 70, 50, 20, 60, 40]。

复杂度：Ologn)。

易错提醒 下沉时必须与两个孩子中更符合堆序的那个交换；最大堆选较大孩子，最小堆选较小孩子。

2.9哈夫曼树与树专题的联系

哈夫曼树既是贪心算法的经典应用，也是带权二叉树的重要考点。它使带权路径长度WPL=∑w l 最小。频率越高的字符通常越靠近根，因此编码越短。

●哈夫曼编码是前缀编码：任一字符的编码都不是另一字符编码的前缀。

●n 个叶子构成的严格二叉哈夫曼树共有2n-1 个结点。

●每次合并的权值之和累加，即可直接得到最终WPL。

2.10高频错误清单

●混淆“结点的度”“树的度”“结点深度”“树的高度”。

● 把no =n_2 +1用到一般树；该公式针对非空二叉树。

●1-based 与0-based 完全二叉树数组下标公式混用。

●认为前序+后序一定能唯一重建二叉树。

●BST 删除双孩子结点时破坏有序性。

● 把AVL 的 LR 型错误地做成一次左旋。

●认为堆的所有元素都是完全有序的，或认为建堆复杂度是O(n log n)。

第三部分 综合题与答题模板

3.1判断一个贪心策略是否正确

典型题10 | 错误策略反例

题目有人提出：活动选择问题总是优先选择持续时间最短的活动。该策略是否一定最优？

精析

38. 要否定一个“总是正确”的策略，只需构造一个反例。

39. 考虑活动A=(0,3)、B=(2,4)、C=(3,6)。B 持续时间最短(2)，选B 后 A、C 都冲突，只能选1个。

40. 若选A 和 C，则可以选2个。因此“最短持续时间优先”不保证最优。

答案 该策略不一定正确。反例中最短活动优先得到1个，而最优解得到2个。

易错提醒 写反例时必须同时给出“贪心结果”和“更优可行解”，只说“可能不对”不能构成完整论证。

3.2遍历序列合法性判断

典型题11 | BST 后序序列判断

题目判断序列1、3、2、6、5、7、4是否可能是某棵BST 的后序遍历。

精析

41. 后序最后一个元素4是根。

42. 根前应先出现所有小于4的左子树元素，再出现所有大于4的右子树元素。序列可分为[1,3,2]与[6,5,7]。

43. 递归检查左段：根为2，可分[1]与[3]，合法。

44. 递归检查右段：根为7，其前[6,5]都小于7；继续以5为根时6属于右侧，合法。

答案 该序列可以是BST 的后序遍历。

复杂度：朴素递归最坏$O(n^2)$；使用边界与单调栈可做到 O(n)。

易错提醒 必须递归检查每个子区间，不能只检查整段相对根的左右划分。

3.3树上递归答题模板

```
Result solve(Node*root){
  if (Iroot)return base_case;        // 空树边界
  ResultL=solve(root->left);          // 左子树答案
  Result  R=solve(root->right);       //右子树答案
  return  combine(root,L,R);          // 合并为当前树答案
```

树题的关键是明确返回值含义。例如求高度时返回“当前子树高度”；判断平衡时可同时返回“是否平衡与高度”，避免重复计算。

典型题12 | 一次遍历判断平衡二叉树

题目设计O(n) 算法判断一棵二叉树是否平衡。

精析

45. 后序遍历，因为父结点的判断依赖左右子树高度。

46. 递归函数返回子树高度；若发现不平衡，则返回特殊值-1。

47. 若任一子树返回-1，或左右高度差超过1，当前也返回-1；否则返回max(L,R)+1。

答案 根函数返回值不为-1时整棵树平衡。

复杂度：每个结点访问一次，时间O(n)，递归栈O(h)。

> 易错提醒 若每个结点都单独调用一次求高度函数，最坏会退化为$O(n^2)$。

```
int heightOrFail(Node*x){
  if(lx)return 0;
  intL=heightOrFail(x->lef);
  if(L==-1)return-1;
  int R=heightOrFail(x->right);
  if (R==-1||abs(L-R)>1)return-1;
  return max(L,R)+1;
}
bool isBalanced(Node*root){
  return heightOrFail(root)!=-1;
}
```

第四部分 考前速查表

4.1贪心算法速查

问题	贪心策略	关键条件	复杂度
活动选择	按结束时间升序，能接就选	目标为活动数量最大	O(n log n)
部分背包	按单位价值降序装入	物品可切分	O(n log n)
哈夫曼	反复合并最小两权	构造最优前缀编码	O(n log n)
Kruskal	从小到大选不成环边	无向连通带权图	O(E log E)
Prim	选当前树到外部的最小边	无向连通带权图	O(E logV)
Dijkstra	确定最小暂定距离顶点	边权非负	O(V+E)logV)

4.2树的公式与操作速查

项目	结论
树的边数	n个结点的非空树有n-1条边
二叉树叶子关系	no =n2+1
第i层最大结点数	2^(i-1)(根为第1层)

项目	结论
高度h的满二叉树结点数	2^h-1
n结点完全二叉树高度	L logzn」+1
遍历顺序	前：根左右；中：左根右；后：左右根；层序：队列
BST	中序有序；查找/插入/删除O(h)
AVL	BF∈{-1,0,1};LL右旋，RR左旋，LR先左后右，RL先右后左
堆	堆顶O(1),插入/删除O(logn),建堆O(n)

4.3 一分钟易错检查

● □题目中的“高度”按边数还是层数定义?

● □区间端点相等是否算冲突?

● □背包物品能否切分?

□图是有向还是无向?边权是否可能为负?

● □数组下标从0还是从1开始?

●□ BST是否允许重复键?若允许，放左侧还是右侧?

●□遍历序列中的结点值是否互不相同?

●□ 复杂度写的是平均情况还是最坏情况?

第五部分 补考自测卷

5.1题目

48.【判断】只要一个问题具有最优子结构，就一定可以使用贪心算法。

49. 【选择】活动选择问题中，保证得到最多兼容活动的经典策略是： A.最早开始 B.持续时间最短 C.最早结束 D.权值最大。

50.【计算】部分背包容量15,物品(重量，价值)为(10,20)、(5,15)、(10,10),求最大价值。

51.【判断】 Dijkstra 可以正确处理含负权边但不含负环的图。

52. 【简答】用交换论证说明活动选择中“最早结束”选择是安全的。

53. 【计算】非空二叉树有12个度为2的结点、5个度为1的结点，求叶子数与总结点数。

54. 【选择】含31个结点的完全二叉树高度(根为第1层)是： A.4 B.5 C.6 D.31。

55. 【遍历】已知前序 ABDCE, 中 序DBAEC, 求后序。

56. 【判断】结点值互不相同时，前序和后序遍历可以唯一确定任意二叉树。

57.【BST】 依次插入8、3、10、1、6、14、4、7、13,写出中序遍历。

58. 【AVL】依次插入10、30、20,属于哪种失衡？旋转后根是谁?

59. 【堆】最小堆[2,5,3,9,7,6]删除堆顶后，写出一种正确调整结果。

5.2参考答案与评分点

题号	答案	关键评分点
1	错误	最优子结构不足以保证贪心可行，还需贪心选择性质。
2	C	按结束时间升序。
3	35	单位价值3、2、1;先取第二件得15,再取第一件10重量得20,总计35。
4	错误	Dijkstra要求边权非负。
5	见评分点	取任一最优解O的首活动x;最早结束活动g满足f(g)≤f(x),用g替换x后不减少后续可选范围，故存在含g的最优解；再归纳剩余子问题。
6	13;30	no =n2+1=13;n=13+5+12=30。
7	B	L log231」+1=5。
8	DBECA	A为根；左子树B-D,右子树C的左孩子为E。
9	错误	只有一个孩子时不能由前序+后序判断其在左还是右。
10	1,3,4,6,7,8,10,13,14	BST中序遍历递增。
11	RL;20	先右旋30,再左旋10;20成为局部根。
12	[3,5,6,9,7]	末元素6移根，与较小孩子3交换后满足最小堆；该结果合法。